

Corso di Ingegneria del Software Deliverable di progetto	2023-2024
---	-----------

“Ingegneria del Software” 2023-2024

Docente: Prof. Angelo Furfaro

MINI-CAD

Data	08/05/2024
Documento	Documento Finale – D3

Team Members		
Nome e Cognome	Matricola	E-mail address
LORENZO CICCONE	219916	CCCLNZ01S29C710W@STUDENTI.UNICAL.IT

Sommario

Non è stata trovata alcuna voce d'indice.

1	
---	--

List of Challenging/Risky Requirements or Tasks

Challenging Task	Date the task is identified	Date the challenge is resolved	Explanation on how the challenge has been managed
Capire cosa tenere e cosa invece modificare dal programma originale, rispettando più possibile quella che è la traccia.			Rileggendo molte volte la traccia ed analizzando i punti chiave del programma esistente, sono riuscito prima tramite schema e dopo tramite codice ad implementare tutto il necessario
Riuscire ad analizzare il comando per la creazione dei gruppi all'interno del parser in maniera dinamica, prendendo il giusto numero di id			Ho risolto utilizzando un ciclo while che andasse all'infinito sulla stringa, che raccoglie gli id nella parentesi finchè non trova una parentesi chiusa
Riconoscere a tempo di esecuzione cosa ci sia scritto dopo il comando ls oppure area o perimetro, utilizzare il giusto comando in caso in cui il prossimo simbolo sia un numero oppure una word.			È bastato creare nell'analizzatore lessicale quello che è un metodo per sbirciare sul prossimo simbolo senza prenderlo veramente, in modo poi da poter utilizzare il getNextInt() o il getNextString() in base a quello che troverò, per poi attivare il comando corretto.
Gestire i gruppi come se fossero degli oggetti semplici			Sono riuscito a gestire la cosa concretizzando quello che è il design pattern composite, creando l'oggetto groupObject, che estende quella che è la AbstractGraphicObject del progetto originale, come anche le classi leaf come CircleObject hanno fatto. In modo da

			utilizzare i gruppi come se fossero oggetti normali
Fare l'undo di comandi come delete o remove group, recuperando un qualcosa di eliminato.			Ho risolto creando quello che si potrebbe definire un vero e proprio backup, ripristinabile all'occorrenza di una undo di qualche comando per la delete. Ho utilizzato una hashmap per gestire quello che è il salvataggio degli oggetti grafici, in modo poi da poterli reperire tramite id contenuto nelle key dell'hashmap.

A. Stato dell'Arte

Il programma consente di interpretare righe di testo scritte all'interno di una textbox posta in fondo all'unica pagina che si presenta dopo l'avvio dello stesso. La lista dei comandi utilizzabili con le sintassi corrette è presente in una finestra pop up che compare una volta cliccato il pulsante 'help'. In base al comando inserito, avvengono le azioni svolte nel pannello bianco sopra. Ci sono tutti i comandi specificati dalla richiesta, quindi di creazione, raggruppamento, spostamento e modifica delle figure. È possibile azionare nuovamente il comando inserito in precedenza tramite il bottone 'redo', sempre in alto a sinistra, insieme al pulsante 'undo' che permette di tornare indietro, qualunque sia stato il comando o i comandi utilizzati, in quanto è in grado di salvare una cronologia di istruzioni per poi ripeterle. Permette tutti i comandi citati nel comando 'help' con 3 tipi di oggetti: circle, rectangle e img (qualsiasi immagine tramite percorso).

B. Raffinamento dei Requisiti

Realizzare un mini interprete di comandi da integrare in una applicazione per la manipolazione di oggetti grafici 2d. Esistono solo 3 tipi di oggetti grafici: rettangoli, cerchi ed immagini.

L'interprete, la cui grammatica EBNF è riportata di seguito, supporta i seguenti comandi, alcuni dei quali sono reversibili (se ne può richiedere l'*undo*).

- Creazione di un nuovo oggetto grafico
- Rimozione di un oggetto o un gruppo di oggetti
- Spostamento di un oggetto o un gruppo di oggetti
- Ridimensionamento di un oggetto o un gruppo di oggetti
- Visualizzazione delle proprietà di un oggetto, un gruppo di oggetti o di un tipo di oggetti
- Creazione di un gruppo di oggetti
- Rimozione di un gruppo di oggetti
- Calcolo area (perimetro)

B.1 Servizi (con prioritizzazione)

- 1) **Creazione di un nuovo oggetto grafico** (importanza: Alta, Complessità: Media): permette la creazione degli oggetti con le specifiche inserite;
- 2) **Rimozione di un oggetto o gruppo di oggetti** (importanza: Alta, Complessità: Facile): permette la rimozione degli oggetti specificati;
- 3) **Creazione di un gruppo di oggetti** (importanza: Alta, Complessità: difficile): permette la creazione di un gruppo astratto che contiene al suo interno gli oggetti che abbiamo selezionato;
- 4) **Spostamento di un oggetto o gruppo di oggetti** (importanza: Alta, Complessità: media): permette lo spostamento in determinate coordinate inserite di oggetti singoli o gruppi di oggetti;
- 5) **Ridimensionamento oggetti o gruppi di oggetti** (importanza: Media, Complessità: bassa): permette di modificare i parametri che riguardano le dimensioni di una figura o di un gruppo di figure.
- 6) **Rimozione gruppo di oggetti** (importanza: Media, Complessità: media): permette di eliminare uno dei gruppi creati in precedenza.
- 7) **Visualizzazione delle proprietà di un oggetto, un gruppo di oggetti o di un tipo di oggetti** (importanza: Bassa, Complessità: media): permette di visualizzare tramite finestra popup le informazioni di oggetti, intere categorie di oggetti o gruppi esistenti
- 8) **Calcolo area (perimetro)** (importanza: Bassa, Complessità: bassa): permette di visualizzare sul terminale l'area della figura scelta, la somma delle aree del tipo selezionato oppure la somma delle aree di tutto. Funziona anche col perimetro.

B

.2 Requisiti non Funzionali

1. Usabilità:
 - a. Interfaccia Utente: Il design dell'interfaccia è minimalistico, creato così intenzionalmente, per agevolare l'utente finale, pratica e diretta;
 - b. Documentazione: nell'output di sistema, ci sono vari messaggi che aiutano a capire cosa si sta facendo e cosa ha dato errore;

2. Performance:

- a. **Tempo di Risposta:** Le operazioni come l'aggiunta, la rimozione e il movimento degli oggetti sono implementate in modo efficiente, suggerendo tempi di risposta brevi. La struttura a mappa per la gestione degli oggetti offre un accesso rapido agli stessi.
- b. **Scalabilità:** L'uso di HashMap per la gestione degli oggetti consente al sistema di scalare efficacemente con un numero elevato di oggetti.

3. Affidabilità:

- a. **Disponibilità:** Non sono stati segnalati crash o problemi di stabilità durante lo sviluppo. L'implementazione robusta delle operazioni suggerisce una buona disponibilità del sistema.
- b. **Gestione degli Errori:** Il codice gestisce situazioni di errore comuni, come il tentativo di rimuovere un oggetto non esistente, con messaggi di errore chiari.

4. Manutenibilità:

- a. **Modularità:** Il codice è strutturato in classi e metodi ben definiti, facilitando la manutenzione e l'espansione futura del sistema.
- b. **Testabilità:** La separazione delle funzionalità in metodi specifici rende il codice facilmente testabile. Ogni operazione può essere testata individualmente.

5. Estensibilità:

- a. **Aggiunta di Nuovi Oggetti:** La struttura del codice, con classi base e gestione degli ID, rende relativamente semplice l'aggiunta di nuovi tipi di oggetti grafici.
- b. **Supporto per Nuovi Comandi:** L'aggiunta di nuovi comandi può essere implementata estendendo le funzionalità esistenti.

B.3 Scenari d'uso dettagliati

Caso D'uso	L'utente vuole creare un nuovo oggetto.
Precondizione	L'applicazione è aperta e l'utente è nella vista principale.
Svolgimento Normale	L'utente scrive nello spazio di testo il comando per la creazione della figura, con i dati della figura richiesti e la posizione nel pannello (x,y). Es: create circle (12.0) (40,50)
Postcondizione	La figura viene creata e mostrata sul pannello bianco.
Descrizione	L'oggetto viene creato, gli viene assegnato un id univoco, inserito in una apposita mappa chiamata objects e poi inviato al GraphicObjectPanel per la visualizzazione.

Caso D'uso	L'utente vuole eliminare un oggetto o un gruppo di oggetti.
Precondizione	L'applicazione è aperta e l'utente è nella vista principale.
Svolgimento Normale	L'utente scrive nello spazio di testo il comando per la eliminazione della figura o gruppo, es: del 1 (1 = id scelto)
Postcondizione	La figura o il gruppo di figure vengono eliminati dal pannello e dalla mappa.
Descrizione	L'oggetto viene cercato tramite id nella mappa, rimosso ed eliminato anche graficamente, dando poi una conferma nel terminale.

Caso D'uso	Creazione Gruppo di oggetti grafici
Precondizione	L'applicazione è aperta e l'utente è nella vista principale. Esistono almeno 2 oggetti.
Svolgimento Normale	L'utente scrive nello spazio di testo il comando per la creazione del gruppo, seguito dagli id degli oggetti da raggruppare
Postcondizione	Il gruppo è stato registrato con successo, ma non è visualizzabile in quanto è visivamente astratto.
Descrizione	Se gli id specificati esistono il sistema crea un nuovo oggetto di tipo GroupObject con id univoco e ci aggiunge gli oggetti con l'id specificato in precedenza nel comando, dopodichè si aggiunge il gruppo alla mappa di oggetti e da conferma nel terminale in caso di successo.

Caso D'uso	Muovere un oggetto grafico
Precondizione	L'applicazione è aperta e l'utente è nella vista principale. Esistono almeno 1 oggetto grafico.
Svolgimento Normale	L'utente scrive nello spazio di testo il comando per lo spostamento degli oggetti grafici, sul pannello gli oggetti si muoveranno nelle posizioni espresse coi comandi.
Postcondizione	Il gruppo di figure o la figura sono in una posizione diversa in quanto è stata aggiornata dal comando.
Descrizione	Se l'id specificato è riconosciuto come id di una figura sola, allora il sistema recupera la posizione dell'oggetto e la aggiorna, stessa cosa avviene se è un gruppo, apre il gruppo, apre gli oggetti singoli e ne aggiorna la posizione in base al comando di movimento selezionato

Caso D'uso	Calcolare L'area o perimetro di un oggetto o di un gruppo di oggetti
Precondizione	L'applicazione è aperta e l'utente è nella vista principale. Esistono almeno 1 oggetto grafico.
Svolgimento Normale	L'utente scrive nello spazio di testo il comando per il calcolo dell'area o del perimetro insieme all'id, avra' il risultato nel terminale.
Postcondizione	L'area é stata calcolata e mostrata all'utente.
Descrizione	Se l'id specificato è riconosciuto come id di una figura sola, allora il sistema recupera i dati necessari in base al tipo di figura per calcolarne l'area o il perimetro, dopo le manda in output, stessa cosa avviene se è un gruppo, apre il gruppo, apre gli oggetti singoli e ne ricava l'area o il perimetro, sommandoli e poi mandandoli in output.

Caso D'uso	Visualizzazione proprieta' oggetti, gruppi di oggetti o tipo di oggetti.
Precondizione	L'applicazione è aperta e l'utente è nella vista principale. Esistono almeno 1 oggetto grafico.
Svolgimento Normale	L'utente scrive nello spazio di testo il comando per la visualizzazione delle proprieta' (ls) ed in seguito l'id del singolo o del gruppo oppure il tipo.
Postcondizione	Le informazioni richieste vengono esibite in una finestra popup.

Descrizione	Funziona come per l'area ma prende tutti i dati, previo controllo del tipo di dato inserito nel comando, in cui si verifica se è un tipo, gruppo oppure figura singola (funziona anche con ls all in cui mostra tutto)
--------------------	--

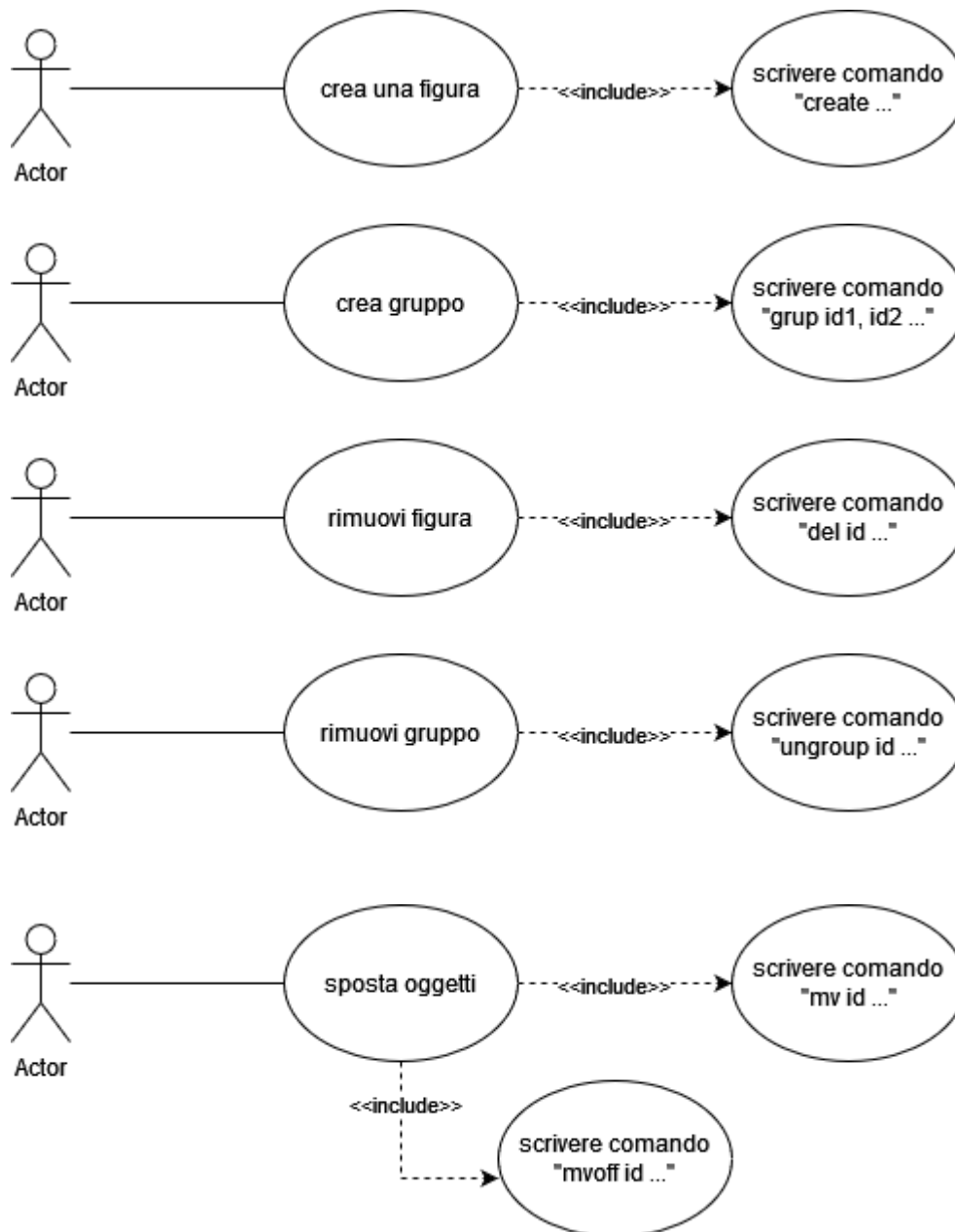
Caso D'uso	Visualizzare la lista di tutti i comandi.
Precondizione	L'applicazione è aperta e l'utente è nella vista principale.
Svolgimento Normale	L'utente preme sul bottone 'help' per ricevere le informazioni che necessita
Postcondizione	Si apre una finestra con tutti i comandi con esempi e spiegazioni.
Descrizione	

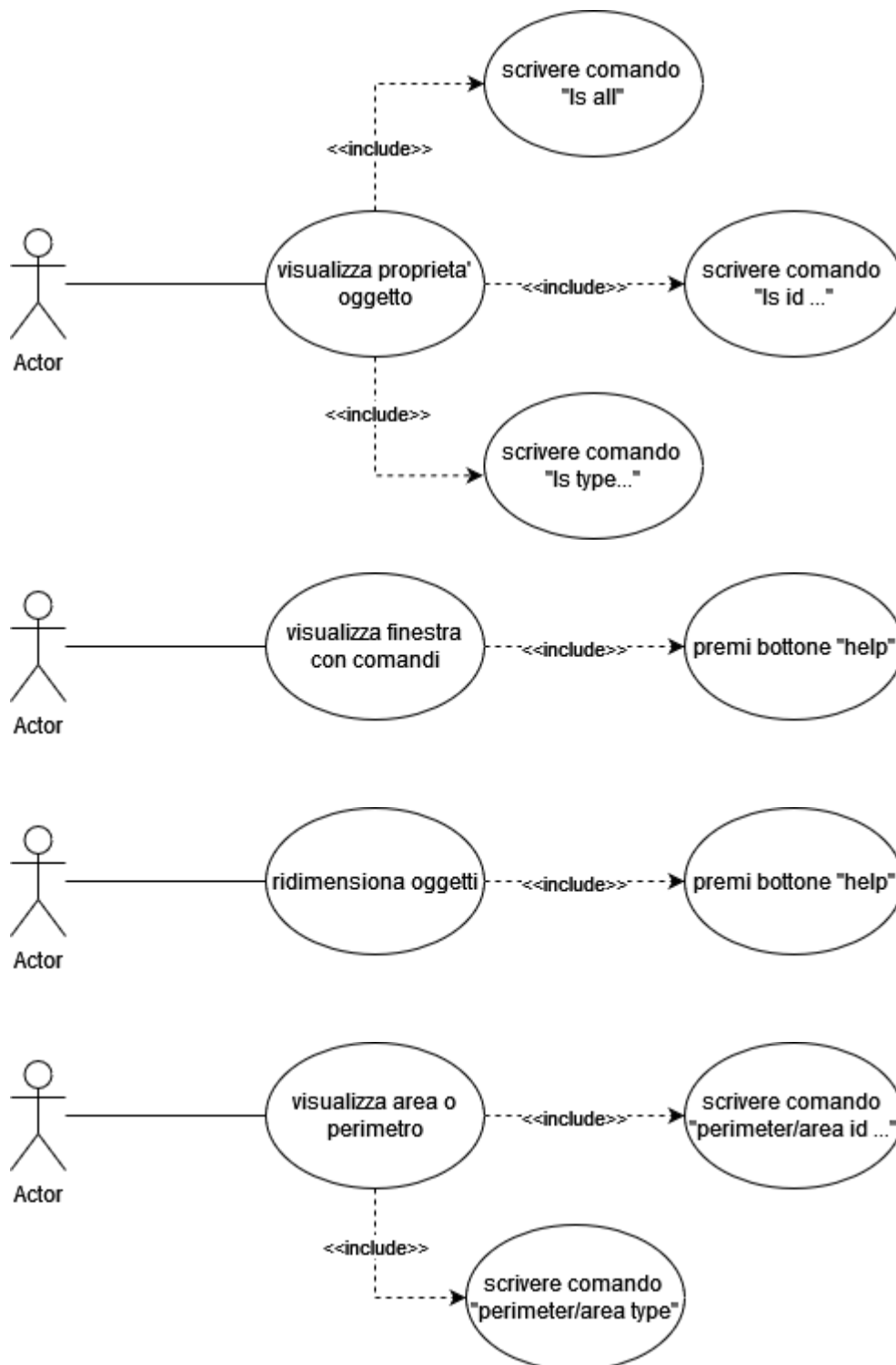
A.5 Assunzioni

La traccia non esprime un numero massimo o minimo di figure che si possono creare, ho quindi deciso di impostare gli id delle figure come numeri sequenziali da 1 a 999 per le figure e da 1000 in poi per i gruppi, per una questione di comodità ed ulteriore sicurezza durante i controlli degli id dei comandi. Esiste quindi un tetto massimo figurativo di 999 figure.

Il pannello non ha dimensioni precise, ho quindi fatto in modo che si potesse allargare tramite mouse ingrandendo la finestra, in modo da poter mettere le figure dove vogliamo con le coordinate.

A.6 Use Case Diagrams

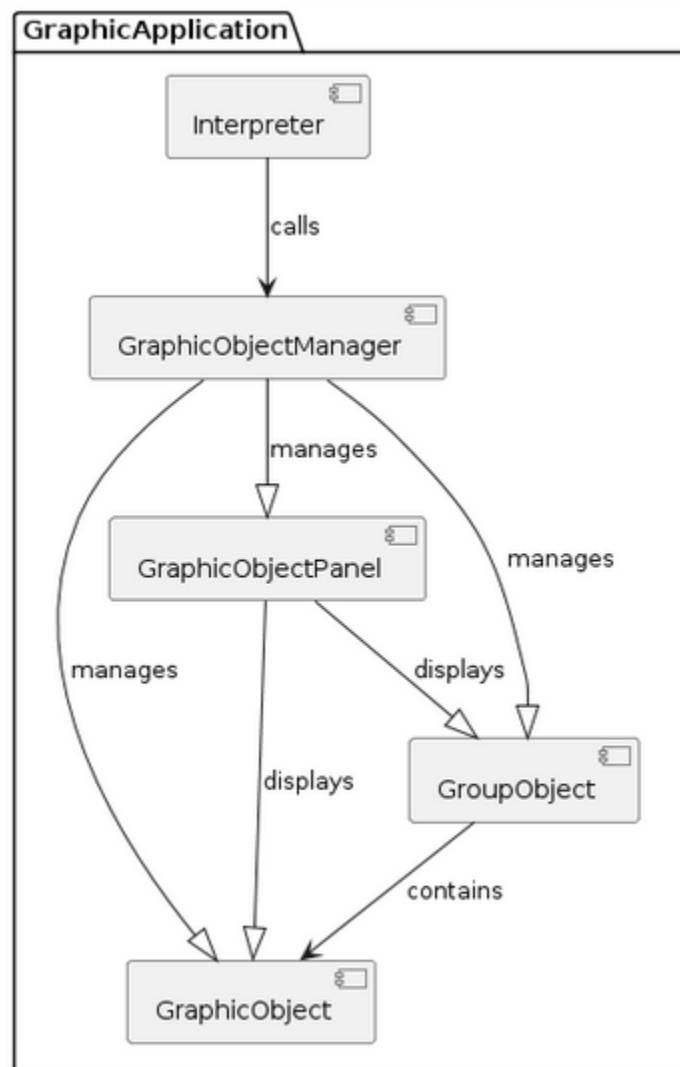




C. Architettura Software

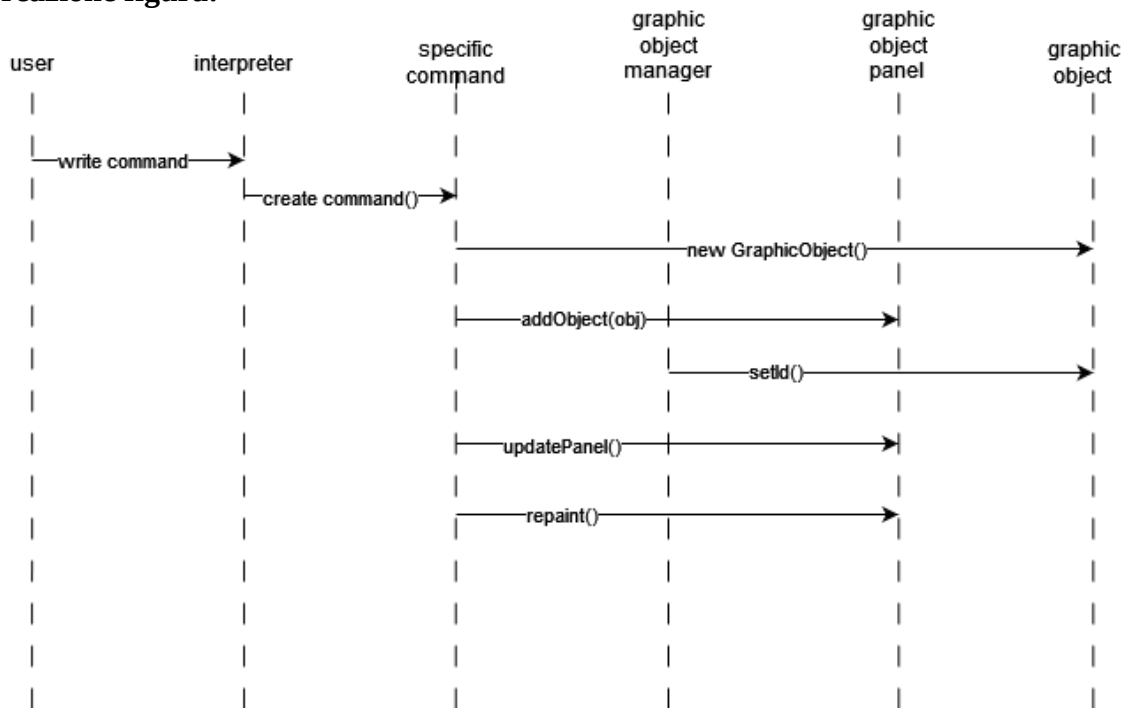
<IF RELEVANT, Report here both the static and the dynamic view of your system design, in terms of a Component Diagram, and their related Sequence Diagrams >

C.1 The static view of the system: Component Diagram

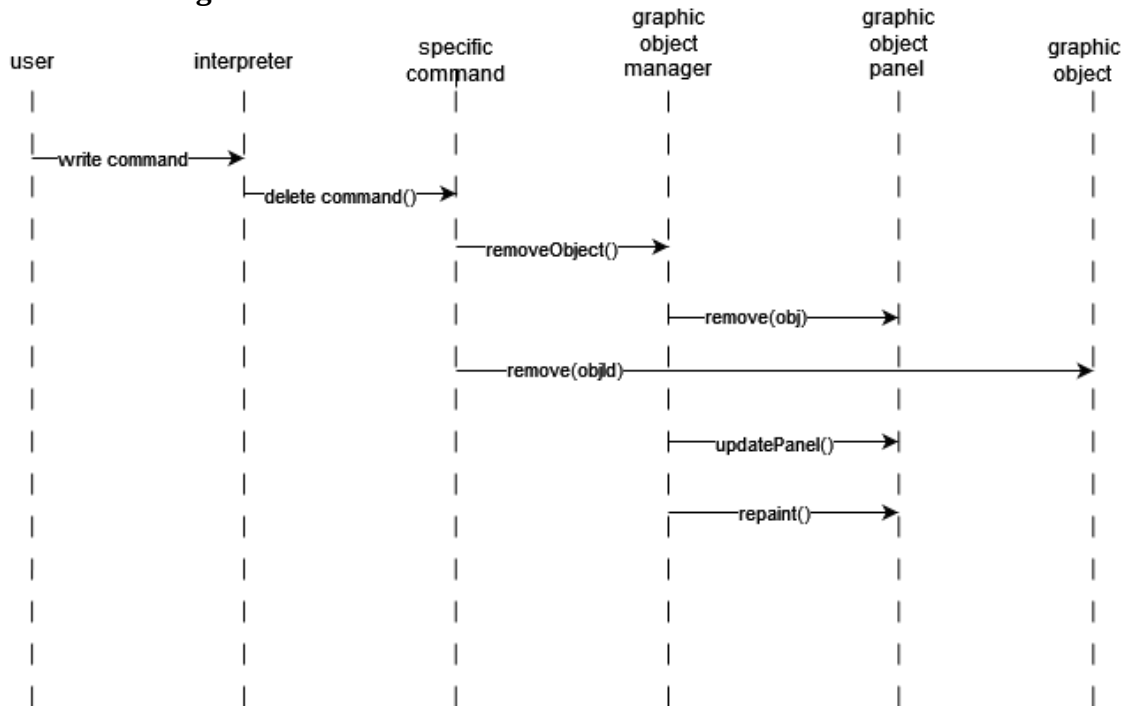


*C.2 The dynamic view of the software architecture: Sequence
Diagram*

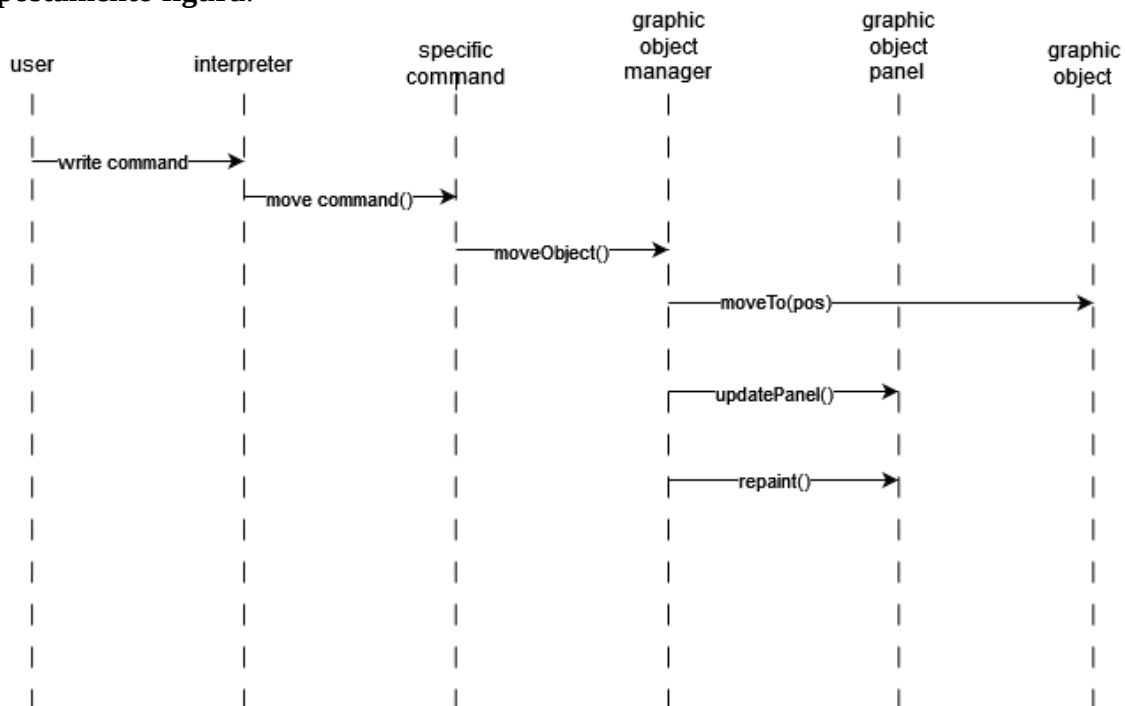
Creazione figura:



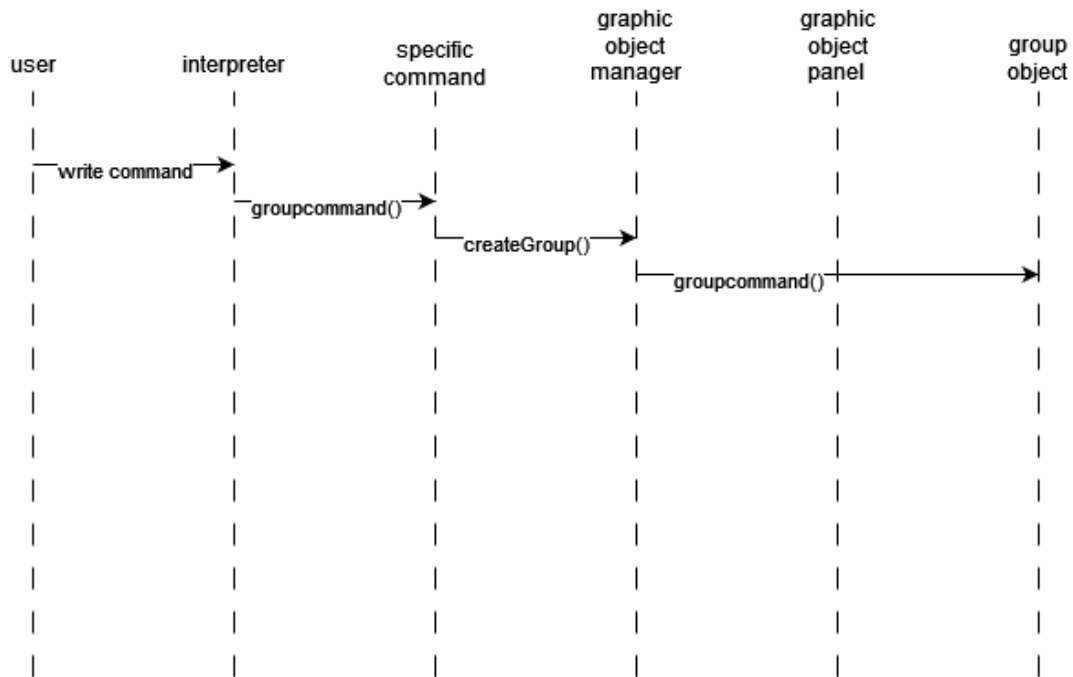
Cancellazione figura:



Spostamento figura:



Creazione Gruppo:



E. Scelte Progettuali (Design Decisions)

COMMAND PATTERN

Descrizione: Incapsula una richiesta come un oggetto, permettendo di parametrizzare i client con richieste diverse, fare logging e implementare undo/redo.

Implementazione:

- **Interfaccia Command:** Command.
- **Concrete Command:** CreateCircleCommand, CreateRectangleObject, CreateImgCommand, MoveCommand, MoveOffsetCommand, ScaleCommand, GroupCommand, UngroupCommand, AreaCommand, PerimeterCommand.
- **Invoker:** HistoryCommandHandler gestisce la cronologia dei comandi

FACTORY METHOD PATTERN

Descrizione: Definisce un'interfaccia per creare oggetti, ma lascia che siano le sottoclassi a determinare quale classe istanziare.

Implementazione:

- **Factory:** GraphicObjectViewFactory fornisce le viste per ogni tipo di oggetto grafico.
- **Concrete Product:**
 - CircleObjectView
 - RectangleObjectView
 - ImageObjectView

COMPOSITE PATTERN

Descrizione: Permette di comporre oggetti in strutture ad albero e trattare gli oggetti singoli e composti in modo uniforme.

Implementazione:

- **Component:** GraphicObject.
- **Composite:** GroupObject, che contiene una lista di GraphicObject.
- **Leaf:**
 - CircleObject
 - RectangleObject
 - ImageObject

OBSERVER PATTERN

Descrizione: Definisce una dipendenza uno-a-molti tra oggetti, in modo tale che quando un oggetto cambia stato, tutti i suoi osservatori vengono notificati.

Implementazione:

- **Subject:** GraphicObject, che notifica i cambiamenti.
- **Observer:** GraphicObjectListener, implementato da GraphicObjectPanel

INTERPRETER PATTERN

Descrizione: Definisce una rappresentazione per la grammatica della traccia e definisce un interprete in grado di interpretare i comandi della textbox.

Implementazione:

- **Analizzatore lessicale:** AnalizzatoreLessicale, che trasforma una sequenza di caratteri in sequenze di token.
- **Parser:** Parser, organizza i token e richiama i metodi del pattern Command.

F. Progettazione di Basso Livello

COMMAND PATTERN

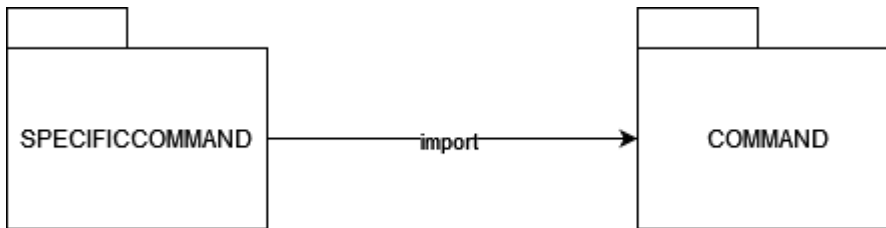
```
CommandHandler.java X
src > command > CommandHandler.java > {} command
1 package command;
2
3 public interface CommandHandler {
4     void handle(Command cmd);
5 }
6

HistoryCommandHandler.java X
src > command > HistoryCommandHandler.java > ...
1 package command;
2
3 import java.util.LinkedList;
4
5 import view.GraphicObjectPanel;
6
7 public class HistoryCommandHandler implements CommandHandler {
8     private int maxHistoryLength = 100;
9
10    private final LinkedList<Command> history = new LinkedList<>();
11    private final LinkedList<Command> redoList = new LinkedList<>();
12
13    GraphicObjectPanel gpanel; "gpanel": Unknown word.
14
15    public HistoryCommandHandler(GraphicObjectPanel panel) {
16        this(maxHistoryLength:100);
17        this.gpanel = panel; "gpanel": Unknown word.
18    }
19
20    public HistoryCommandHandler(int maxHistoryLength) {
21        if (maxHistoryLength < 0)
22            throw new IllegalArgumentException();
23        this.maxHistoryLength = maxHistoryLength;
24    }
25
26    public void handle(Command cmd) {
27        if (cmd.doIt()) {
28            addToHistory(cmd);
29        } else {
30            history.clear();
31            if (redoList.size() > 0)
32                redoList.clear();
33        }
34
35        public void redo() {
36            if (redoList.size() > 0) {
37                Command redoCmd = redoList.removeFirst();
38                redoCmd.doIt();
39                gpanel.repaint(); "gpanel": Unknown word.
40                history.addFirst(redoCmd);
41            }
42        }
43
44        public void undo() {
45            if (history.size() > 0) {
46                Command undoCmd = history.removeFirst();
47                undoCmd.undoIt();
48                gpanel.repaint(); "gpanel": Unknown word.
49                redoList.addFirst(undoCmd);
50            }
51        }
52
53        private void addToHistory(Command cmd) {
54            history.addFirst(cmd);
55            if (history.size() > maxHistoryLength) {
56                history.removeLast();
57            }
58        }
59    }
60 }
```

```
Command.java X
src > command > Command.java > ...
1 package command;
2
3 public interface Command {
4
5     boolean doIt();
6
7     boolean undoIt();
8 }
9

NaiveCommandHandler.java X
src > command > NaiveCommandHandler.java > {} command
1 package command;
2
3 public class NaiveCommandHandler implements CommandHandler {
4
5     @Override
6     public void handle(Command cmd) {
7         cmd.doIt();
8     }
9
10 }
11 }
```

Il pattern composite mi è stato molto d'aiuto per memorizzare tutti i comandi a cui era possibile fare poi un redo oppure un undo, grazie alla funzione doIt ed UndoIt è stato molto piu' semplice ed ordinato creare dei metodi per annullare alcune azioni, che molto spesso non sono affatto banali come una delete di un gruppo. Le classi specifiche per i comandi che implementano Command, sono all'interno del package Specificcommand, esiste quindi una classe per ogni comando, con doIt ed UndoIt.



FACTORY METHOD PATTERN

```
src > view > GraphicObjectViewFactory.java X
1 package view;
2
3 import java.util.HashMap;
4 import java.util.Map;
5
6 import shapes.GraphicObject;
7
8 public enum GraphicObjectViewFactory {
9
10     FACTORY;
11
12     private final Map<Class? extends GraphicObject>, GraphicObjectView> viewMap = new HashMap<>();
13
14     GraphicObjectView createView(GraphicObject go) {
15         return viewMap.get(go.getClass());
16     }
17
18     public void installView(Class? extends GraphicObject> clazz, GraphicObjectView view) {
19         viewMap.put(clazz, view);
20     }
21 }
22
```

```
src > view > CircleObjectView.java X
1 package view;
2
3 import java.awt.Graphics2D;
4 import java.awt.geom.Ellipse2D;
5 import java.awt.geom.Point2D;
6
7 import shapes.CircleObject;
8 import shapes.GraphicObject;
9
10 public class CircleObjectView implements GraphicObjectView {
11
12     @Override
13     public void drawGraphicObject(GraphicObject go, Graphics2D g) {
14         CircleObject co = (CircleObject) go;
15         Point2D position = co.getPosition();
16         double r = co.getRadius();
17         double x = position.getX() - r;
18         double y = position.getY() - r;
19         g.draw(new Ellipse2D.Double(x, y, r * 2.0, r * 2.0));
20     }
21 }
22
```

```
src > view > ImageObjectView.java X
1 package view;
2
3 import java.awt.Graphics2D;
4 import java.awt.Image;
5 import java.awt.geom.Dimension2D;
6 import java.awt.geom.Point2D;
7
8 import shapes.GraphicObject;
9 import shapes.ImageObject;
10
11 public class ImageObjectView implements GraphicObjectView {
12
13     @Override
14     public void drawGraphicObject(GraphicObject go, Graphics2D g) {
15         ImageObject io = (ImageObject) go;
16         Dimension2D dim = io.getDimension();
17         Point2D position = io.getPosition();
18         Image image = io.getImage();
19         int w = (int) (dim.getWidth());
20         int h = (int) (dim.getHeight());
21         int x = (int) (position.getX() - w / 2);
22         int y = (int) (position.getY() - h / 2);
23
24         g.drawImage(image, x, y, w, h, observer=null);
25     }
26 }
27
```

```
src > view > RectangleObjectView.java X
1 package view;
2
3 import java.awt.Graphics2D;
4 import java.awt.geom.Rectangle2D;
5 import shapes.GraphicObject;
6 import shapes.RectangleObject;
7
8 public class RectangleObjectView implements GraphicObjectView {
9
10     @Override
11     public void drawGraphicObject(GraphicObject go, Graphics2D g) {
12         if (go instanceof RectangleObject) {
13             RectangleObject rect = (RectangleObject) go;
14
15             double x = rect.getPosition().getX();
16             double y = rect.getPosition().getY();
17             double width = rect.getWidth();
18             double height = rect.getHeight();
19
20             g.draw(new Rectangle2D.Double(x, y, width, height));
21         }
22     }
23 }
24
```

COMPOSITE PATTERN

```
GroupObject.java X
src > composite > GroupObject.java > {} composite
9  import shapes.GraphicObject;
10
11  public class GroupObject extends AbstractGraphicObject {
12      private List<GraphicObject> members = new ArrayList<>();
13      private int id;
14
15      public GroupObject(int id) {
16          this.id = id;
17      }
18
19      public void addMember(GraphicObject obj) {
20          members.add(obj);
21      }
22
23      public List<GraphicObject> getMembers() {
24          return new ArrayList<>(members); // Ritorno una copia per prevenire modifiche esterne "Ritorno": Unknown word.
25      }
26
27      @Override
28      public void moveTo(Point2D p) {
29          for (GraphicObject member : members) {
30              member.moveTo(p);
31          }
32      }
33
34      @Override
35      public void scale(double factor) {
36          for (GraphicObject member : members) {
37              member.scale(factor);
38          }
39      }
40
41      @Override
42      public void setId(int id) {
43          this.id = id;
44      }
45
46      @Override
47      public int getId() {
48          return this.id;
49      }
50
51      @Override
52      public Point2D getPosition() {
53          // TODO Auto-generated method stub
54          throw new UnsupportedOperationException(message:"Unimplemented method 'getPosition'");
55      }
56  }
```

GroupObject non è un oggetto normale come sono ad esempio circle, rectangle o image, è un oggetto che è in grado di “incapsularne” altri semplici, quelli scritti sopra, immagazzinandoli in una struttura dati di tipo ArrayList, contenente appunto solo oggetti di tipo GraphicObject.

INTERPRETER PATTERN

La classe Parser è responsabile dell'interpretazione dei comandi analizzati dalla classe AnalizzatoreLessicale, che utilizza uno streamtokenizer per riconoscere lettere, simboli, numeri ecc.. tramite il metodo prossimoSimbolo. Il parser quindi inizializza l'analizzatore lessicale e gestisce i comandi in base a cosa trova scritto, indirizzando poi il tutto tramite i metodi appositi come parseMoveCommand, ParseCreateCommand ecc... che servono poi a richiamare le classi dentro il package Specificcommand.

G

. Spiegare come il progetto
soddisfa i requisiti funzionali (FRs)
e quelli non funzionali (NFRs)

Requisiti funzionali:

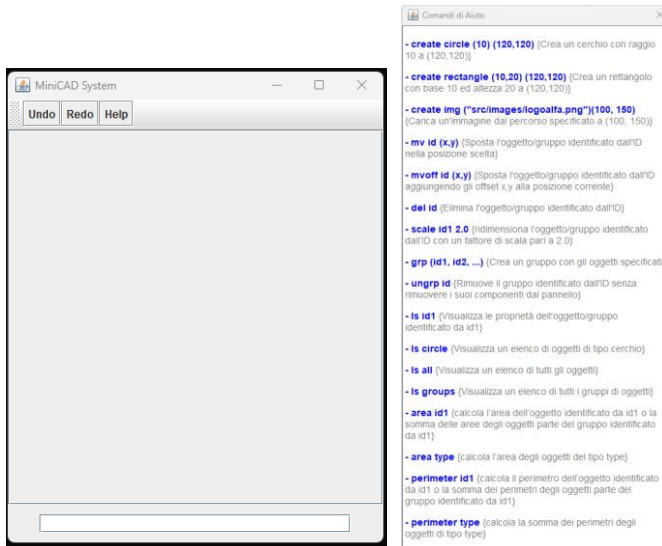
Sono stati tutti soddisfatti tramite i comandi appositi, elencati nella finestra che si apre tramite pulsante “help” nella interfaccia grafica.

Requisiti non funzionali:

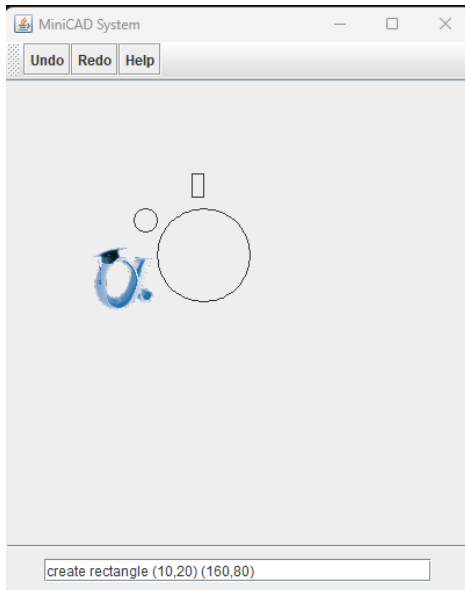
- **Usabilita’:** l’interfaccia molto semplice ed intuitiva garantisce una esperienza utente semplice anche per chi non ha esperienza nel campo, è un buon mix che rende semplice l’utilizzo sia ad un utente esperto che non, grazie anche al bottone help che aiuta a chiarire eventuali dubbi sulle funzionalita dei comandi o sulla sintassi stessa.
- **Performance:** il sistema sfrutta l’inserimento in apposite hashmap, questo permette di essere leggero e rapido, qualsiasi comando è istantaneo, con un tempo di risposta immediato.
- **Affidabilità:** il sistema è stato testato in moduli, quindi ogni parte è stata posta a diversi test pratici e di stress, risolvendo qualsiasi tipo di bug trovato. Il sistema risulta quindi difficile da bloccare o far crashare.
- **Manutenibilità ed Estensibilità:**
l’ordine dei vari package e della struttura complessiva, fa si che la manutenzione e l’eventuale aggiunta di un nuovo oggetto, risulti semplice e veloce. Il tutto testato, in quanto ho inserito prima img e circle, una volta finito tutto ho aggiunto il rectangle, è bastato aggiungere la classe per l’oggetto e quella della view, automaticamente tutti i metodi funzionano senza bisogno di adattarsi a nuovi aggiornamenti.

Appendix. Prototype

Questa è come si presenta l'interfaccia grafica assieme alla finestra help:



Questa è come si presenta l'interfaccia con alcuni oggetti creati sopra:



Corso di Ingegneria del Software Deliverable di progetto	2023-2024
---	------------------